

xln

*Финансы как система
проверяемых машин.*

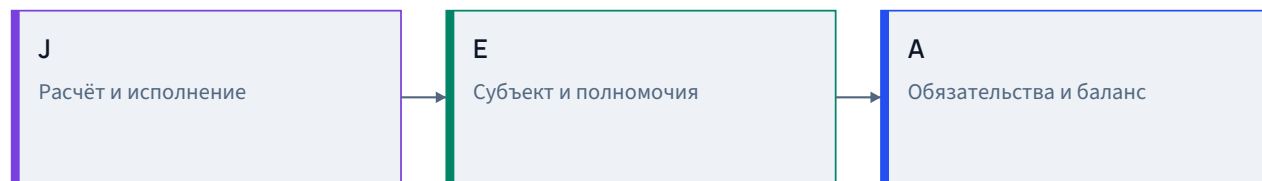


FIG. 01 J/E/A - финансовая модель. Runtime исполняет и сохраняет её реплики.

Основные модули, протоколы и инварианты ядра.
Объяснение по исходному коду с числовыми примерами.

SUBJECT xln · Jurisdiction / Entity / Account

BASELINE d6a0845fd96c · main · 2026-10-06

AUDIENCE Инженеры, архитекторы и финансовые институты

Карта руководства

I · ФИНАНСОВАЯ МОДЕЛЬ

J/E/A: язык финансов	3
Четыре границы исполнения	4
Карта модулей ядра	5
State, Replica и Input	6

II · RUNTIME И ENTITY

Runtime: сначала WAL	7
Outputs, доставка и reject	8
Entity: право действовать	9
Entity consensus: replay	10
Три стадии финансового frame	11

III · ACCOUNT И ОПЕРАЦИИ

Account: единая ось RCPAN	12
Capacity: сколько можно отправить	13
Bilateral frame и ACK	14
Commitments: что подписано	15
Путь прямого платежа	16
HTLC и Paybook	17
Orderbook и same-J swap	18
Cross-J: связанные обязательства	19
Кредит, lending и организация	20

IV · J И НАДЁЖНОСТЬ

J adapters: внешний факт	21
J settlement и sealed batch	22
Спор: исполнить доказательство	23
Delta Transformers	24
Storage и recovery	25
Workers и TS / Rust parity	26
Проверка и границы гарантий	27

Словарь и маршрут по исходникам

[SOURCE docs/core/j-jedi-architecture.md](#)

CHAPTER 01

J/E/A: язык финансов

Расчётная власть, экономический субъект и двустороннее обязательство.

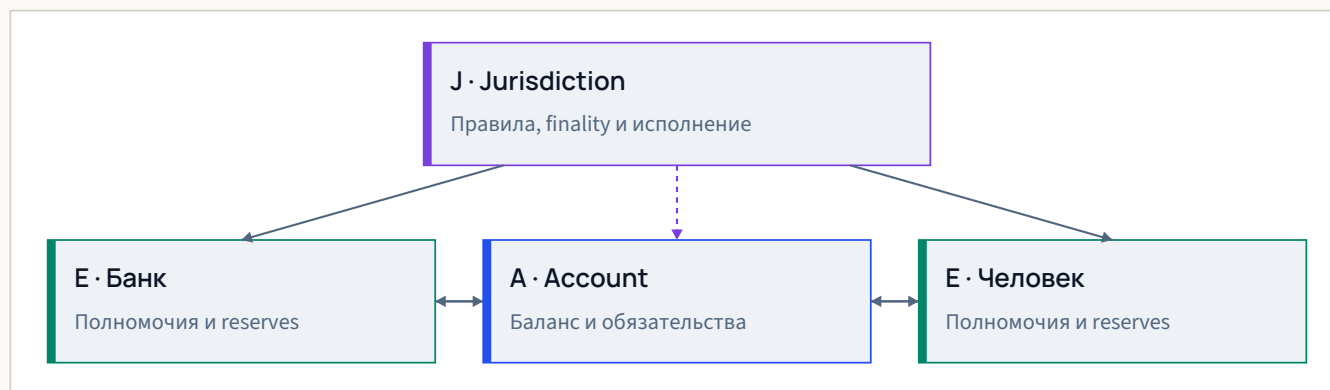


FIG. 02

J задаёт расчётную среду. E вступают в отношения A друг с другом.

Три роли уже существуют в экономике

Jurisdiction определяет, какое доказательство признаётся и как обязательство исполняется. Entity определяет, кто вправе действовать от имени человека или организации. Account фиксирует отношения двух Entity: активы, обеспечение, кредит и условные обязательства. xln использует эти роли как общую модель финансов.

Что делает модель программируемой

Правила перехода становятся явными; участник может воспроизвести результат и проверить подписанное состояние. Merkle commitments позволяют связывать доказательства с точными данными. Репликация даёт независимую проверку. Открытый доступ зависит также от правил конкретной J, а исполнимость - от её расчётных механизмов.

Роль	Финансовый смысл	В xln
J	Расчёт и исполнение	Depository + EntityProvider + adapters
E	Полномочия и политика	Entity consensus + Hanko
A	Баланс и обязательства	RCPAN + bilateral frames

SCOPE

J/E/A - модель. Подключение центрального банка или иной традиционной расчётной системы потребует её собственного адаптера, правил доступа и признания доказательств.

CHAPTER 02

Четыре границы исполнения

Один формат перехода. Разные источники полномочий и моменты commit.

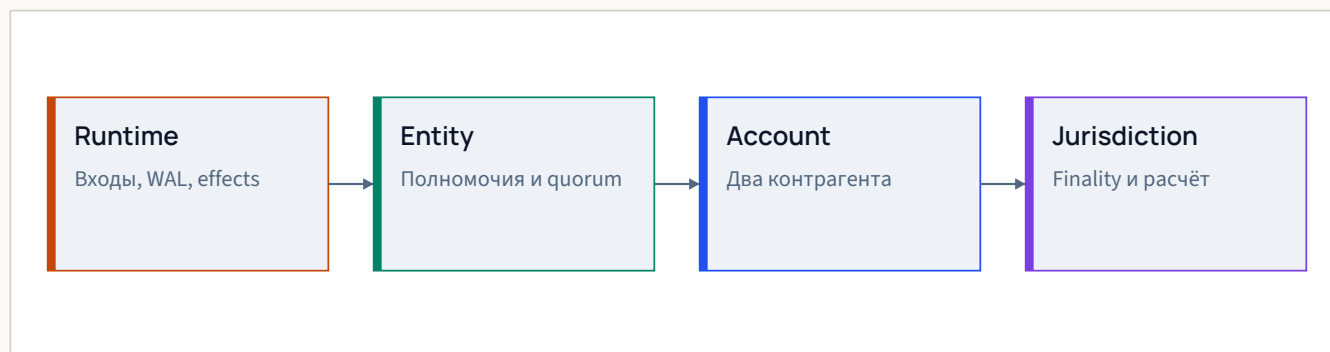


FIG. 03 Runtime размещает реплики Е/А и связывает их с J. Стрелки показывают координацию, а не общий консенсус.

Общая форма

Каждая машина получает свою реплику и точный input, затем возвращает новую реплику и упорядоченные outputs. Input управляет одной машиной. Дочерний AccountInput проходит через Entity без подмены подписанного payload. Сетевые отправки и обращения к J выполняет Runtime после durable commit.

Общий словарь сохраняет различия

Runtime имеет одного writer. Entity требует сертификат своего board. Account согласует двусторонний frame/ACK. J подтверждает внешнее исполнение. Поэтому одинаковые имена State, Input, Tx и Frame не означают общий reducer: у каждого уровня своя граница доверия.

Уровень	Кто подтверждает	Что считается завершением
R	Локальный writer	WAL commit
E	Board с weighted threshold	Сертифицированный EntityFrame
A	Два контрагента	Bilateral frame/ACK
J	Выбранная J	Аутентифицированная finality

CHAPTER 03

Карта модулей ядра

Владение логикой видно по каталогам и по направлению зависимостей.

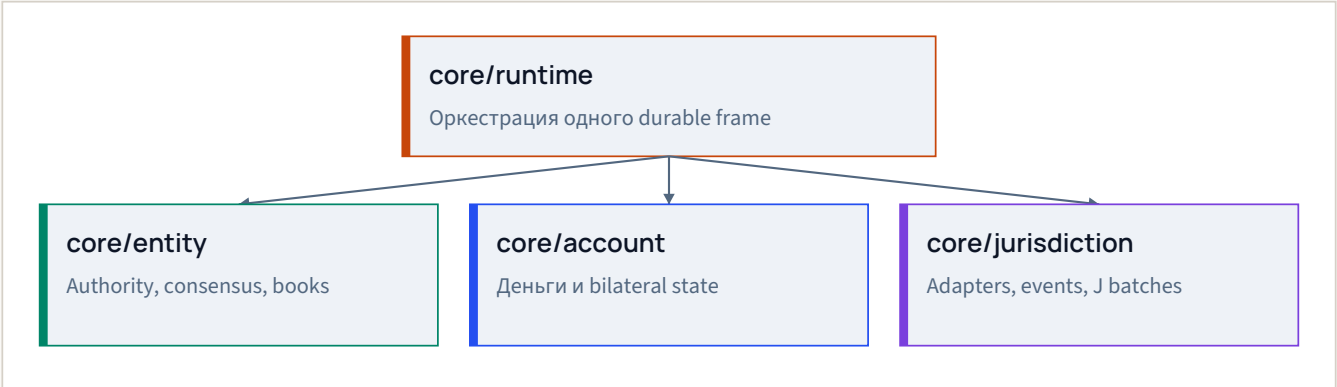


FIG. 04 Финансовые изменения принадлежат Account handlers. Entity допускает и маршрутизирует; Runtime сохраняет.

Как читать код

Начинайте с конкретного входа: RuntimeInput, EntityInput или AccountInput. Найдите его validator, переход и владельца outputs. Затем проследите момент, когда candidate становится committed. Для денежной операции закончите чтение в Account handler и deriveDelta; для внешнего расчёта - в sealed batch и контракте J.

Каталог	Ответственность
core/runtime/	Intake, frame, outbox, delivery, lifecycle
core/entity/	Board, commands, consensus, Paybook, Book intents
core/account/	RCPAN, Tx handlers, frames, settlement proofs
core/jurisdiction/	Наблюдение J, нормализация событий, submission
core/hanko/; core/protocol/	Подписи, codecs, Patricia, proof domains
core/storage/; core/network/	WAL/recovery и аутентифицированный транспорт
core/orderbook/; core/rscore/	Matcher и engine/worker boundaries
core/watchtower/; core/api/	Recovery support и внешняя API поверхность

SOURCE [core/runtime/frame/process.ts](#) | [core/entity/consensus/frame/application.ts](#) | [core/account/tx/apply.ts](#)

CHAPTER 04

State, Replica и Input

Финансовое состояние и координация следующего шага имеют разных владельцев.

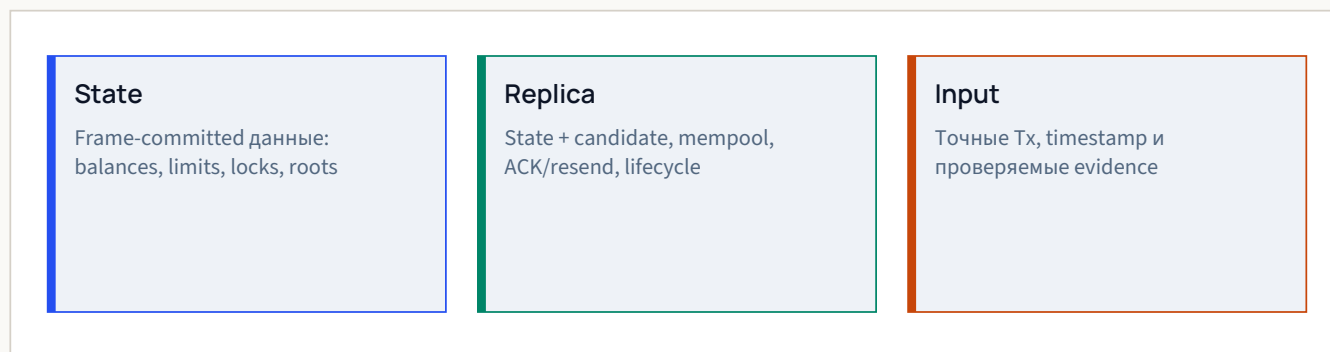


FIG. 05 Replica - живой контейнер. Machine - логика перехода, а не ещё один интерфейс данных.

Детерминизм начинается на входе

Одинаковые предыдущая реплика и input должны дать одинаковое новое состояние и outputs. Внутри перехода используются управляемые timestamps и подготовленная инфраструктурная evidence. Live RPC, wall-clock timers и незафиксированная случайность не могут становиться скрытым источником финансового решения.

Вложенный каскад

RuntimeInput несёт RuntimeTx и адресованные EntityInput. EntityInput несёт EntityTx и consensus evidence. EntityTx.accountInput содержит точный дочерний AccountInput. Локальный финансовый intent создаёт AccountTx[] и входит в тот же applyAccountInput через локальный admission; его не отправляют в P2P как поддельный AccountInput.

Порядок - часть результата

Accepted inputs сохраняют плотные позиции. Outputs каждого input сохраняют естественный порядок. Worker completion time не меняет эту последовательность. Сортировка допустима там, где она задана codec или commitment; произвольная сортировка денежных outputs создаёт другой протокол.

RECOVERY

Координационные поля не получают отдельного durable журнала. Нужная текущая replica восстанавливается из канонического checkpoint и принятых Runtime WAL inputs.

CHAPTER 05

Runtime: сначала WAL

Внешний мир видит только результат уже сохранённого Runtime frame.

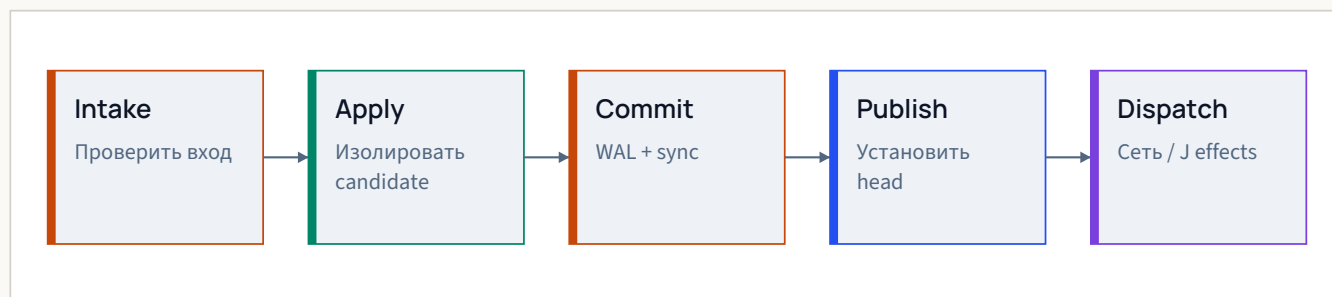


FIG. 06 Канонический путь `applyAndCommitRuntimeFrame`. Side effects следуют за durable boundary.

Что происходит в одном frame

Runtime выбирает точный input, проверяет admission и строит candidate. Дочерние transitions создают state changes, events и output plan. Затем Runtime связывает их с commit identity и вызывает сохранение. Только после этого committed frame публикуется, а его outputs превращаются в сетевые сообщения или J actions.

Почему порядок критичен

Если сообщение отправить раньше WAL, peer может подписать финансовое продолжение состояния, которое отправитель потеряет при сбое. Если WAL уже committed, повторная доставка допустима: Account умеет распознавать точный дубликат. Повторный apply того же платежа недопустим.

Место сбоя	Что известно	Правильная реакция
До commit	Candidate не durable	Discard либо halt/reload по классу ошибки
После commit	Frame окончательный	Восстановить head и доставить outbox
Commit outcome неясен	Нельзя угадывать	Fail-stop с evidence; проверить durable head

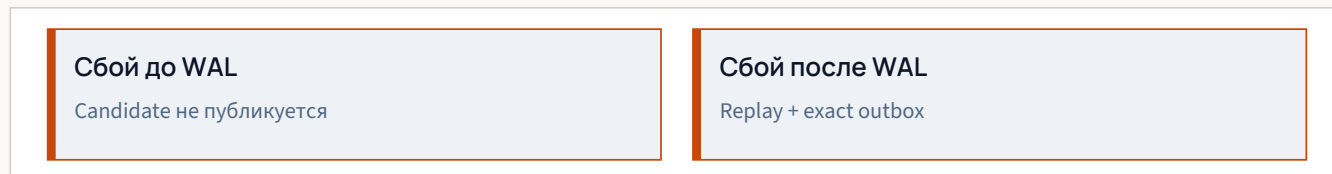


FIG. 07 Один crash boundary меняет допустимый следующий шаг.

SOURCE `core/runtime/frame/process.ts` | `core/storage/commit/commit.ts`

CHAPTER 06

Outputs, доставка и reject

Транспорт доставляет evidence. Финансовое подтверждение делает Account.

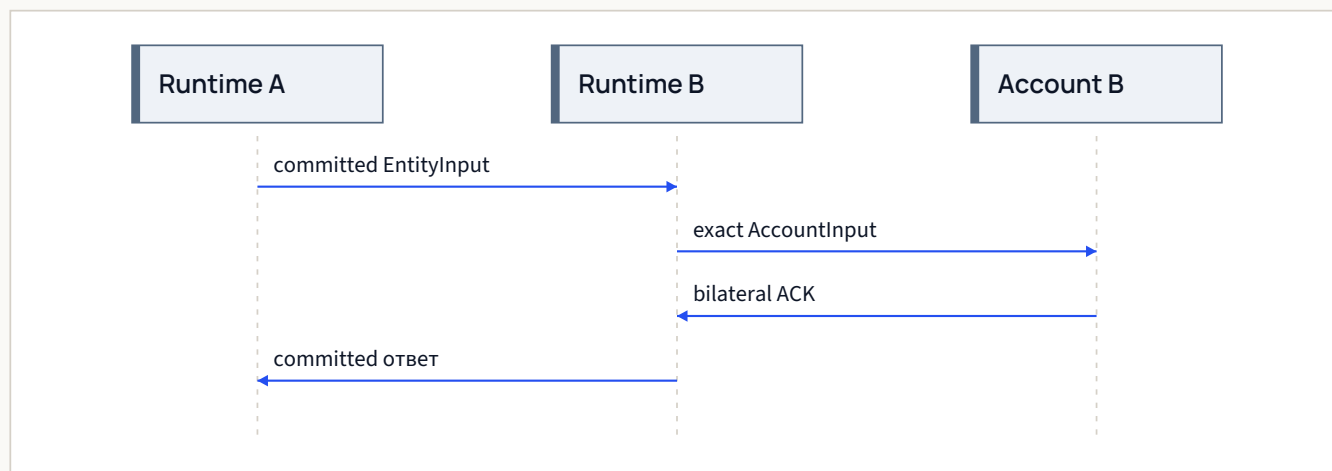


FIG. 08

Сетевое получение пакета и bilateral ACK обозначают разные события.

От адреса Entity к адресу реплики

Application output определяет destination Entity и payload. Runtime в candidate превращает это в точный signer-addressed EntityInput. Уже адресованное validator message сохраняет signer. Delivery может объединять совместимые tx-only envelopes, сохраняя исходную финансовую последовательность.

Плохой input не равен сломанному Runtime

Sender-caused ошибка получает typed reject внутри transition. Runtime loop применяет fail-fast либо log-and-drop вне машины: replay не зависит от NODE_ENV. Каноническая гранулярность - конкретная отвергнутая Tx, а не вся очередь честного signer. Storage или consensus invariant failure требует fail-stop с причиной и evidence.

AUTHORITY

Session MAC, шифрование и envelope signature защищают доставку. Они не заменяют Hanko, Account stateHash или bilateral ACK. В ws-protocol прямо запрещён второй transport receipt, который мог бы считать финансовый outbox завершённым.

CHAPTER 07

Entity: право действовать

Hanko связывает действие с конкретной организацией и её полномочиями.

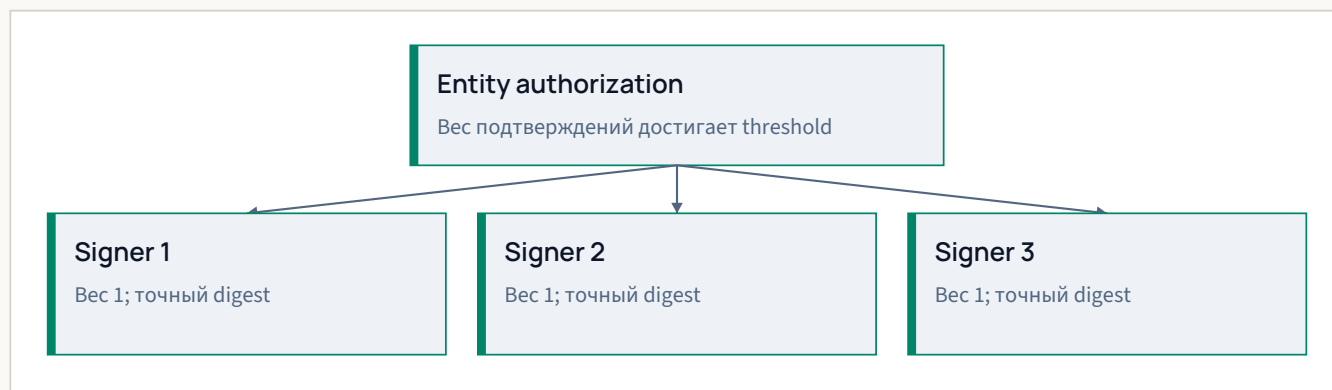


FIG. 09

Пример board 2-of-3. Повтор одной подписи не даёт второй голос.

Полномочия отделены от транспорта

Entity может представлять человека, компанию или иной субъект. Его board задаёт members, weights и threshold. Hanko упаковывает подписи и claims, по которым проверяется, кто именно авторизовал digest. Проверка использует сертифицированное board authority, а не конфигурацию, которую произвольно прислал peer.

Подпись относится к точному действию

EntityFrame hash, AccountFrame hash, dispute proof и J batch имеют разные назначения. Quorum подписывает соответствующие secondary hashes; witnesses затем входят в их точные payloads. Подпись одного объекта нельзя использовать как согласие на другой. Индивидуальный entityCommand дополнительно связан с command nonce; коллективное действие проходит свой proposal path.

Смена board - тоже протокол

Старые доказательства остаются привязаны к авторитету своей эпохи. Обновление board и Account Hanko refresh требует явной сертифицированной связи. Иначе смена ключей могла бы сделать законное старое состояние непроверяемым либо дать новому ключу право переписать прошлое.

```
weight(signer 1) + weight(signer 2) = 2 >= threshold 2
```

FIG. 10

Для board 2-of-3 две разные допустимые подписи достигают threshold.

SOURCE [core/hanko/signing.ts](#) | [core/hanko/claims.ts](#) | [core/entity/command/index.ts](#)

CHAPTER 08

Entity consensus: replay

Validator подтверждает самостоятельно вычисленный frame, а не обещание proposer.

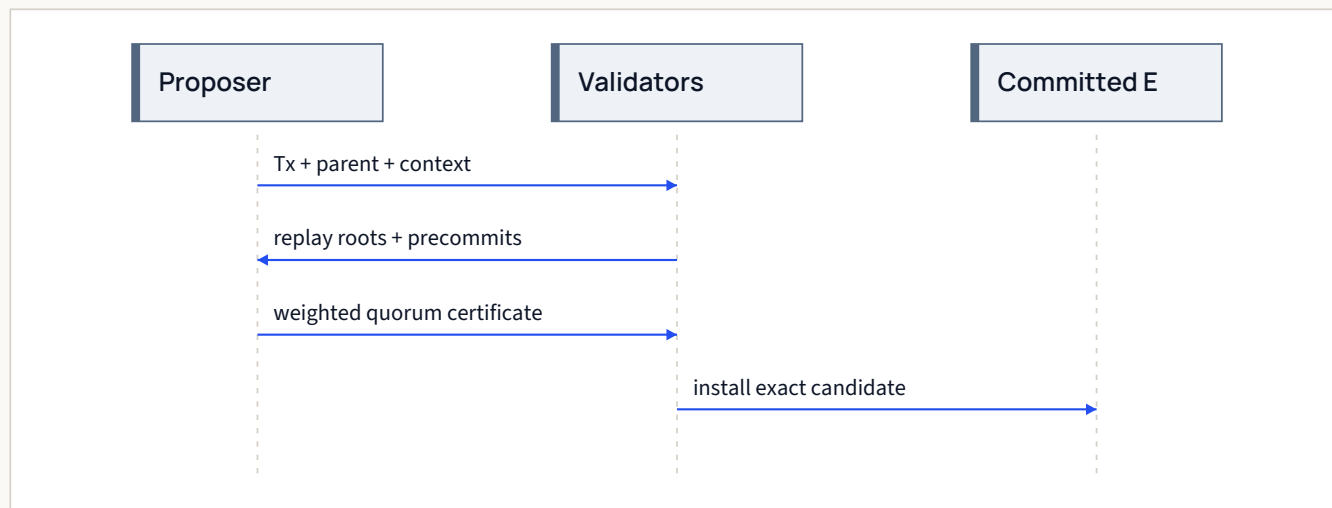


FIG. 11 Candidate не заменяет committed EntityState до сертификации.

Что связывает EntityFrame

Frame содержит height, parentFrameHash, timestamp, entityContext, точные Tx, events, stateRoot и authorityRoot. Validator воспроизводит переход и сравнивает свои commitments с proposal. Secondary manifest связывает подписи с Account, dispute и J payloads, созданными именно этим frame.

Повтор, конфликт и отставание

Точный повтор уже известного frame распознаётся по height/hash и не добавляет состояние. Другой hash той же позиции - конфликт, требующий явного rejection. Catch-up должен подтвердить lineage и authority. Leader view и timeout evidence управляют тем, кто вправе продолжить proposal; время прихода WebSocket пакета не задаёт порядок Entity history.

ONE PATH

Single-signer Entity использует тот же candidate/certificate/commit pipeline. Его локальный threshold достигается сразу; финансовая семантика остаётся общей.

CHAPTER 09

Три стадии финансового frame

Inbound Account state должен попасть в Entity candidate до работы Books.

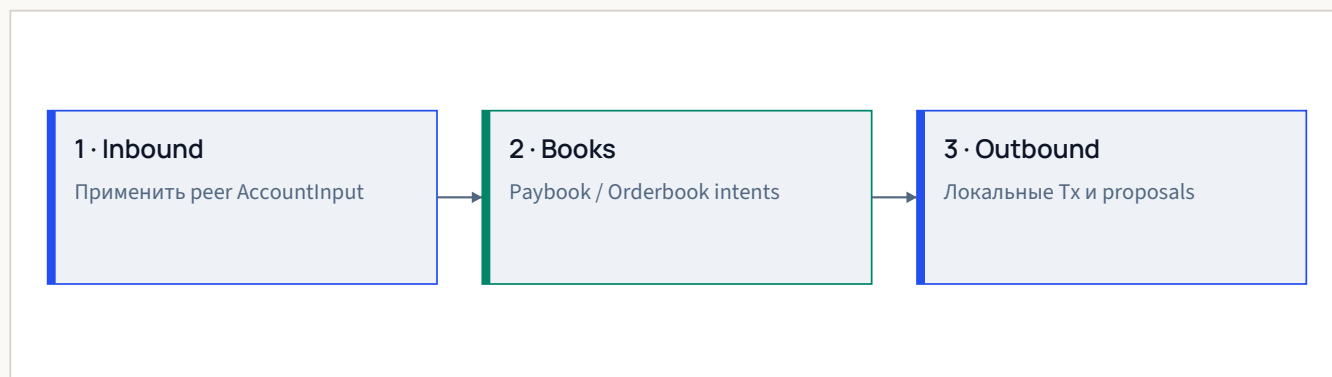


FIG. 12

Между стадиями есть зависимости. Параллельные workers сохраняют эту последовательность.

Сначала факт, затем решение

Stage 1 обрабатывает принятые Account inputs. Их committed child state материализуется в точный Entity candidate. Stage 2 работает с этим состоянием: Books применяют intents и подготавливают финансовую работу. Stage 3 допускает локальные AccountTx и строит outbound proposals после завершения Books.

Books координируют, Account меняет деньги

Paybook связывает lock, секрет, направление и продолжение платежа. Orderbook выбирает совместимые orders и создаёт settlement work. BookIntent slots удерживают происхождение и порядок результатов. Денежная часть всё равно проходит Account transition, capacity validation и bilateral certification.

Где проходит worker boundary

Account stage принимает exact inputs и возвращает упорядоченные results, изменённые subroots и post-account evidence. Dirty roots могут оставаться не sealed до последней стадии. Создание второй поверхности Account state или слияние Books с outbound нарушило бы общую последовательность и условия replay.

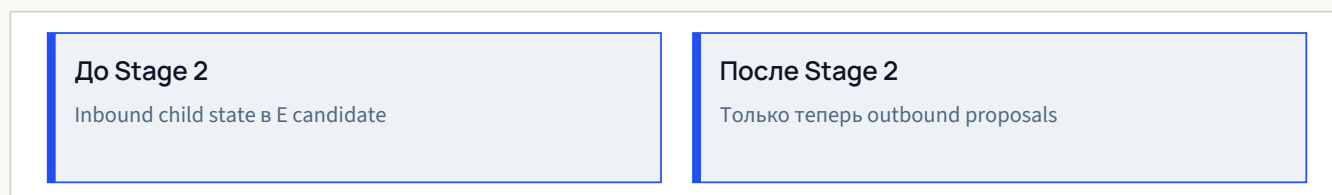


FIG. 13

Books обязаны видеть результат входящих Account переходов.

SOURCE `core/entity/consensus/frame/application.ts` | `core/entity/books/book-intents.ts` | `core/rscore/ts-worker/protocol.ts`

CHAPTER 10

Account: единая ось RCPAN

Обеспечение и кредит описываются одним двусторонним балансом.

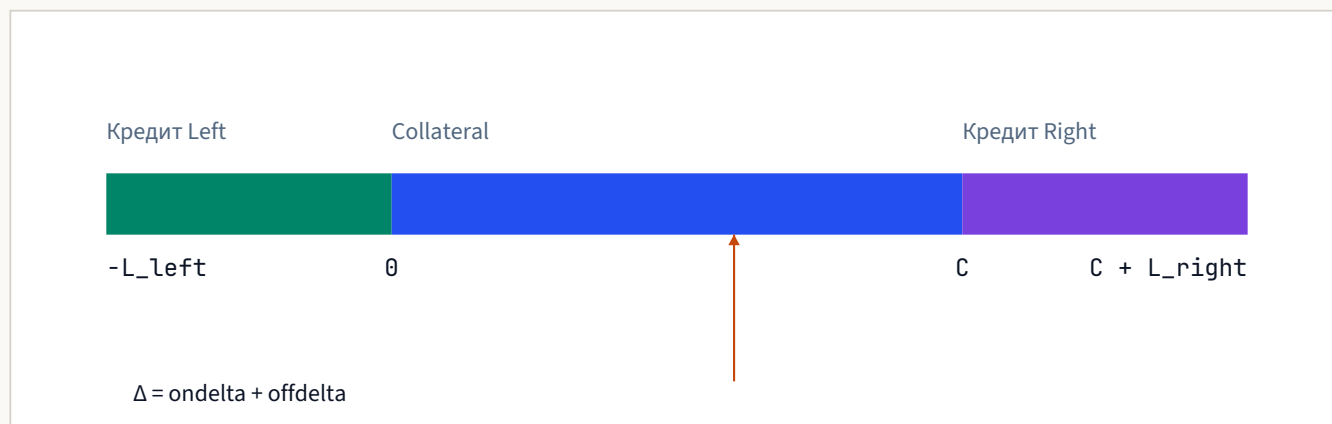


FIG. 14

Области RCPAN показаны схематически. Фактические размеры зависят от C и двух лимитов.

Инвариант допуска

Для новых допустимых операций: $-L_left \leq \Delta \leq C + L_right$. C - общий collateral, Δ - позиция в канонической перспективе Left. Left определяется лексикографически меньшим Entity id. Right предоставляет Left лимит L_left ; Left предоставляет Right лимит L_right . По умолчанию кредит равен нулю.

Что означает положение Δ

При $0 \leq \Delta \leq C$ обеспечение делится: Left получает Δ , Right получает $C - \Delta$. При $\Delta < 0$ Left должен Right величину $-\Delta$, а collateral относится Right. При $\Delta > C$ Right должен Left величину $\Delta - C$, а collateral относится Left. Один и тот же Account допускает полностью обеспеченные и кредитные отношения.

Область	Получает Left	Получает Right
$\Delta < 0$	Долг Left: $-\Delta$	Collateral C + требование к Left
$0 \leq \Delta \leq C$	Collateral Δ	Collateral C - Δ
$\Delta > C$	Collateral C + требование к Right	Долг Right: $\Delta - C$

LIMIT CHANGE

Снижение лимита ограничивает новый риск, но не стирает ранее подписанный долг. Номинальный диапазон допуска и историческая задолженность должны читаться отдельно.

CHAPTER 11

Capacity: сколько можно отправить

Баланс, доступный кредит и уже занятые обязательства дают разные величины.

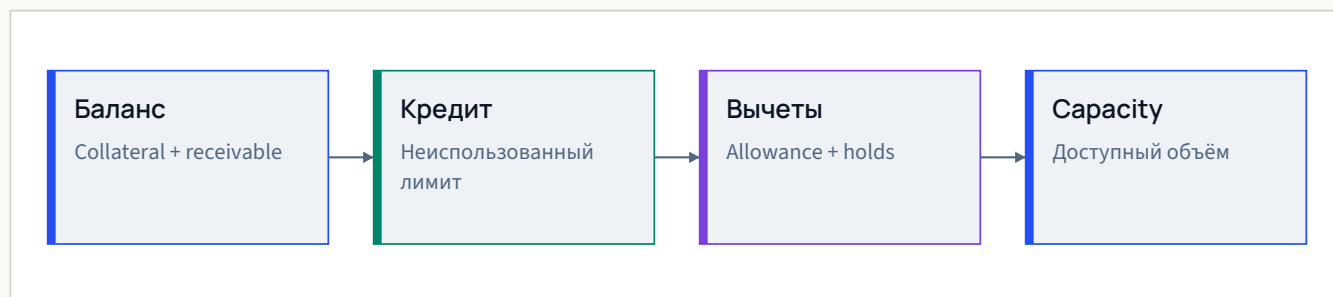


FIG. 15 Это разложение deriveDelta. Holds не дают обещать одну и ту же ликвидность дважды.

Числовой пример

Пусть $C = 100$, $\Delta = 40$, $L_left = 30$, $L_right = 50$. У Left есть 40 collateral и 30 неиспользованного кредита. Если $leftHold = 10$, $leftAllowance = 0$, то Left может отправить 60. У Right есть 60 collateral и 50 кредита; при нулевых вычетах он может отправить 110.

Знак платежа

Left платит Right 20: Δ уменьшается с 40 до 20. Right платит Left 20: Δ увеличивается с 40 до 60. Перспективу пользователя меняет `deriveDelta(delta, isLeft)`; сами данные Account остаются каноническими. UI, routing и lending должны использовать ту же функцию.

Перспектива	Collateral	Кредит	Hold	outCapacity
Left	40	30	10	60
Right	60	50	0	110

CREDIT EXAMPLE

$C = 0$, оба лимита = 3. При $\Delta = 0$ Left платит 2: $\Delta = -2$, Left должен 2. Затем Right платит 3: $\Delta = 1$, Right должен 1. Подписанные встречные требования меняют позицию без нового collateral.

До	Left 40	Right 60
После	Left 20	Right 80

FIG. 16 $C = 100$. Платёж Left → Right на 20: доля Left 40 → 20.

SOURCE `core/account/utils.ts` | `core/account/tx/handlers/balance/direct-payment.ts`

CHAPTER 12

Bilateral frame и ACK

Контрагент проверяет переход собственным исполнением exact Tx.

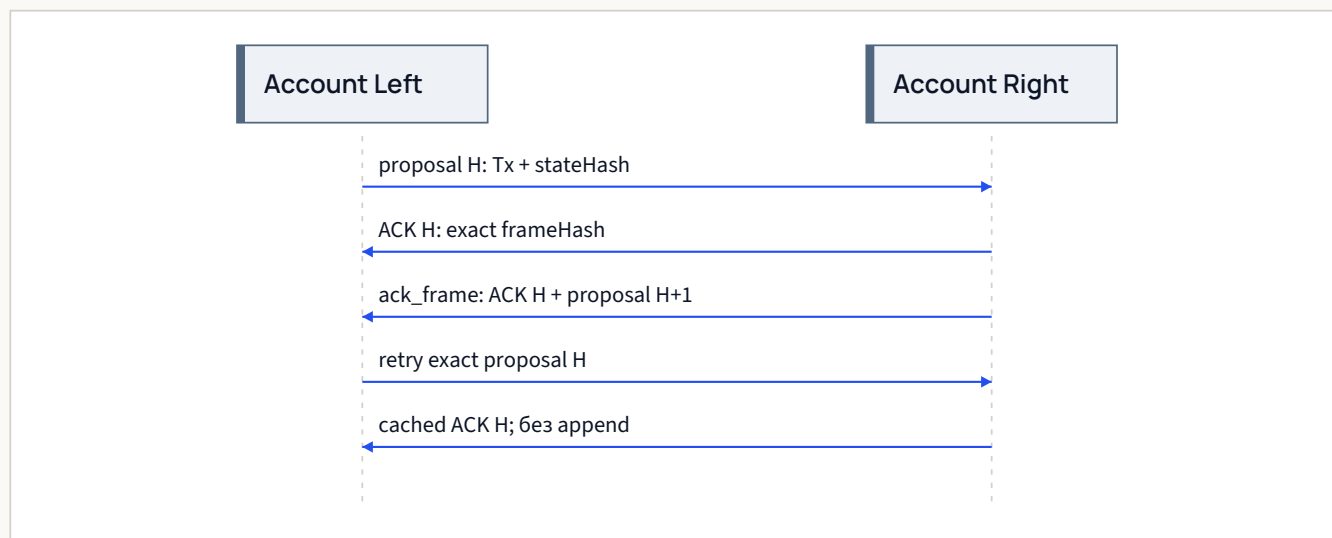


FIG. 17

АСК и следующий proposal относятся к разным эпохам состояния и сохраняют отдельные commitments.

Одна входная поверхность

`applyAccountInput` принимает `peer frame`, `ack`, `ack_frame`, `dispute` и `board_hanko_refresh`. Локальный `enqueue` и `authenticated external_finality` входят в этот же владелец `transition` через свои внутренние типы. `Peer proposal` проходит `envelope validation`, проверку `height/hash/signature` и воспроизведение `AccountTx`.

Точный повтор не двигает деньги

Повтор того же `proposal` или АСК возвращает каноническую `cached evidence` без нового `append`. Несовпадение `hash`, `height` или подписи - явный конфликт. Отдельный `replay protection` нужен для `signed intents` и `J proofs`: `Account frame height` не заменяет каждый другой `nonce`.

FAILURE CASE

Если АСК потерян, `proposer` сохраняет незавершённую координацию и повторяет `exact evidence`. Нельзя считать платёж `committed` по одному факту отправки `proposal` или создавать новый платёжный `payload` для `retry`.

CHAPTER 13

Commitments: что подписано

Root полезен только вместе с точной проекцией, domain и правилами encoding.

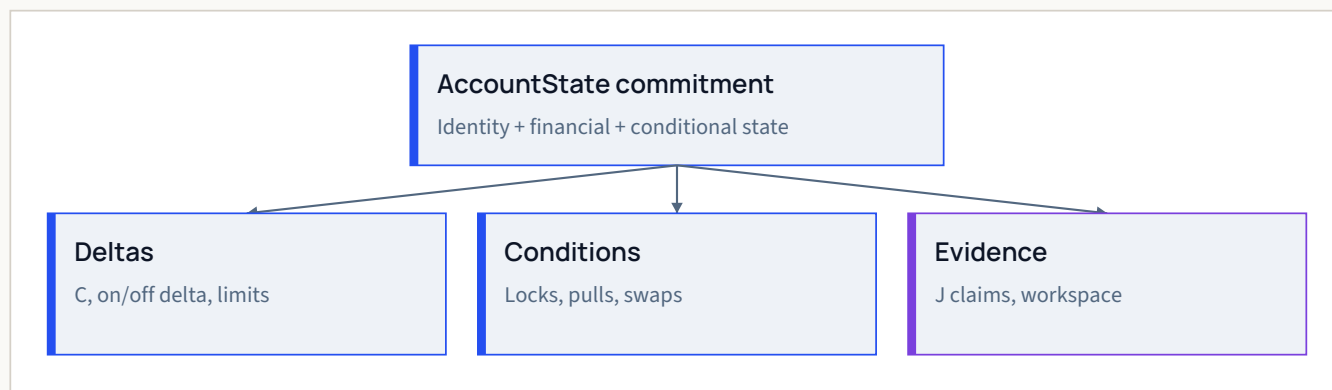


FIG. 18

Пример состава bilateral state. Account envelope имеет отдельную область владения.

Domain исключает чужую расчётную среду

Account state связывает Left, Right и domain из chainId + depositoryAddress. Одинаковый ticker или человекочитаемая jurisdiction label не создаёт одинаковый актив. AccountInput обязан совпасть с канонической идентичностью Account и его расчётной конфигурацией.

Integrity root и proof root выполняют разные задачи

state-root.ts использует flat integrity digest для небольшого набора целиком сравниваемых sections и сохраняет Merkle share там, где нужны кескак proofs. Растущие коллекции имеют собственные typed persistent commitments. Поэтому словосочетание «Merkle state» не означает, что каждый digest - один и тот же тип дерева или принимаемый J proof.

Hash не заменяет проверку

Root связывает данные, но не доказывает корректность перехода сам по себе. Его проверяют через exact decode, authority verification и replay. Для спора дополнительно нужен тот proof format, который понимает J contract; одного локального diagnostic hash недостаточно.

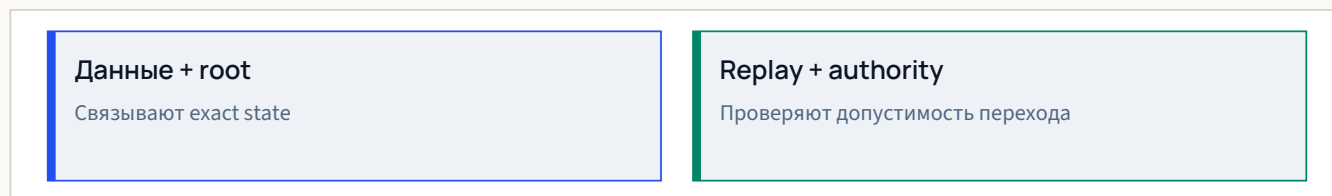


FIG. 19

Commitment и доказательство законности перехода дополняют друг друга.

SOURCE `core/account/commitment/state-root.ts` | `core/account/commitment/account-state-value.ts`

CHAPTER 14

Путь прямого платежа

Один intent проходит authorization, admission, bilateral certification и WAL.

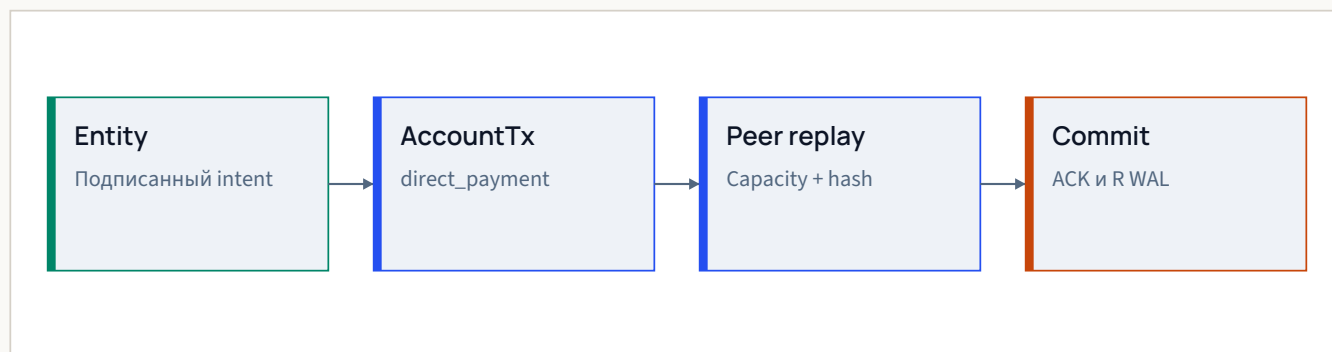


FIG. 20

Стрелки обозначают обязательные этапы; конкретные сообщения могут занимать несколько Runtime frames.

Кто вправе платить

direct-payment.ts выводит payer из авторизованного frame proposer. fromEntityId и toEntityId в wire data проверяются как утверждения о платеже, но сами по себе не дают право списания. direct route имеет один Account hop; trusted forwarding дополнительно связывает объявленный gateway и конечного получателя.

Что меняется

Handler проверяет amount, token и route, затем берёт outCapacity через deriveDelta. Успешный переход изменяет offdelta со знаком, заданным стороной proposer. Изменения живут в изолированном draft. При отказе draft отбрасывается; частичного денежного обновления не остаётся.

Что видит приложение

Account outputs появляются из канонического transition. Entity преобразует их в продолжение работы либо event. Runtime сохраняет frame и лишь затем публикует effect. Forward не должен отправиться из rejected или ещё не committed candidate: иначе следующий Account получил бы деньги, не закреплённые на предыдущем.

EXAMPLE

Left $\Delta = 40$ отправляет Right 20. После допустимого двустороннего перехода $\Delta = 20$. Точный повтор сообщения оставляет $\Delta = 20$; повторный запрос нового платежа требует нового допустимого intent.

CHAPTER 15

HTLC и Paybook

Условный платёж связывает каждый hop одним проверяемым секретом.

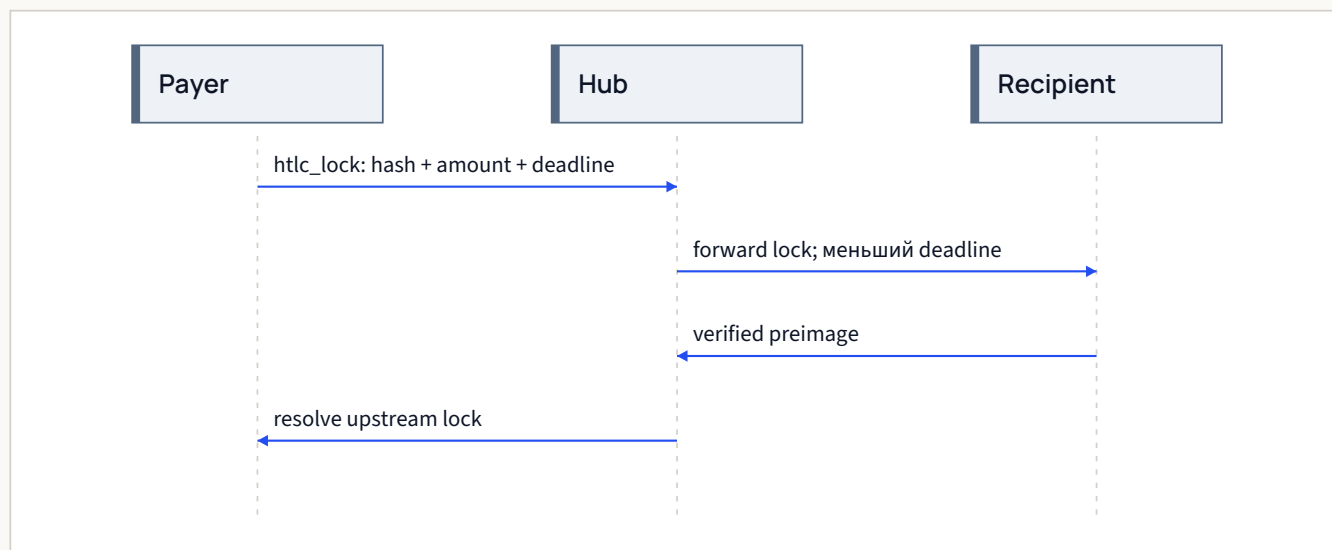


FIG. 21 Показан логический маршрут. Каждый Account hop имеет собственные capacity и bilateral commit.

Подготовка и replay

Route discovery, secret generation и onion construction выполняются при подготовке proposer context. Validators проверяют публичные route/profile/domain evidence, exact debit и fees, deadlines и форму ciphertext. Они воспроизводят зафиксированную подготовку вместо новой случайной маршрутизации.

Почему Paybook отдельный

Account хранит locks и их денежные последствия. Entity Paybook связывает hashlock с inbound/outbound Entity, amount, token и проверенным secret. lockId должен совпадать с hashlock. Конфликт secret, amount или endpoint отвергается. Scheduler выводит wake из frame-controlled deadlines; secret ACK имеет отдельный timeout.

LIVENESS

Наличие preimage ещё требует правильной доставки и commit разрешения. Holds резервируют capacity до исхода lock. Timeout и dispute paths нужны для потерянных сообщений и offline peer; успешная сеть не заменяет эти ветви.

CHAPTER 16

Orderbook и same-J swap

Matcher определяет сделку. Account transitions исполняют её финансовую часть.

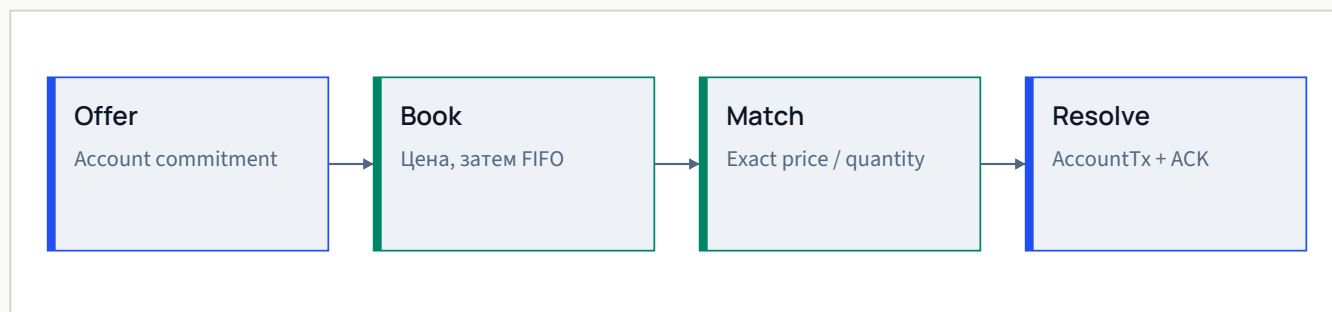


FIG. 22 Book event TRADE - результат matching. Завершённая финансовая операция требует Account settlement.

Один matcher

core/orderbook/core.ts реализует pure price-page limit book. Bids обходятся от лучшей высокой цены, asks - от низкой. Внутри цены сохраняется FIFO по page sequence и slot. Canonical liquidity хранится в radix page trees; RAM order locators восстанавливаются из pages и не становятся отдельным durable источником истины.

От offer к обязательству

Подписанный swap offer входит в Account state и занимает соответствующую capacity. Entity admission выводит executable Book projection. Matcher учитывает eligibility maker и выдаёт exact trade terms. Результат материализуется через Account-owned swap transitions. Cancel тоже должен согласовать подписанное состояние и удаление Book projection.

Слой	Что гарантирует	Чего ещё требует сделка
Book	Price-time выбор и exact quantity	Допустимое финансовое исполнение
Account	Offer, holds, swap resolve	Bilateral certification
Runtime	Durable ordered outputs	Доставка продолжений

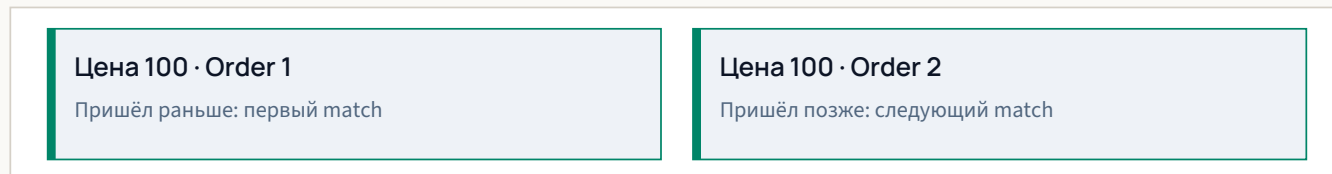


FIG. 23 Одинаковая цена сохраняет FIFO; скорость worker не задаёт приоритет.

SOURCE [core/orderbook/core.ts](#) | [core/orderbook/pages/page.ts](#) | [core/entity/tx/handlers/account/orderbook/index.ts](#)

CHAPTER 17

Cross-J: связанные обязательства

Две J сохраняют свои часы; signed Pull связывает settlement evidence.

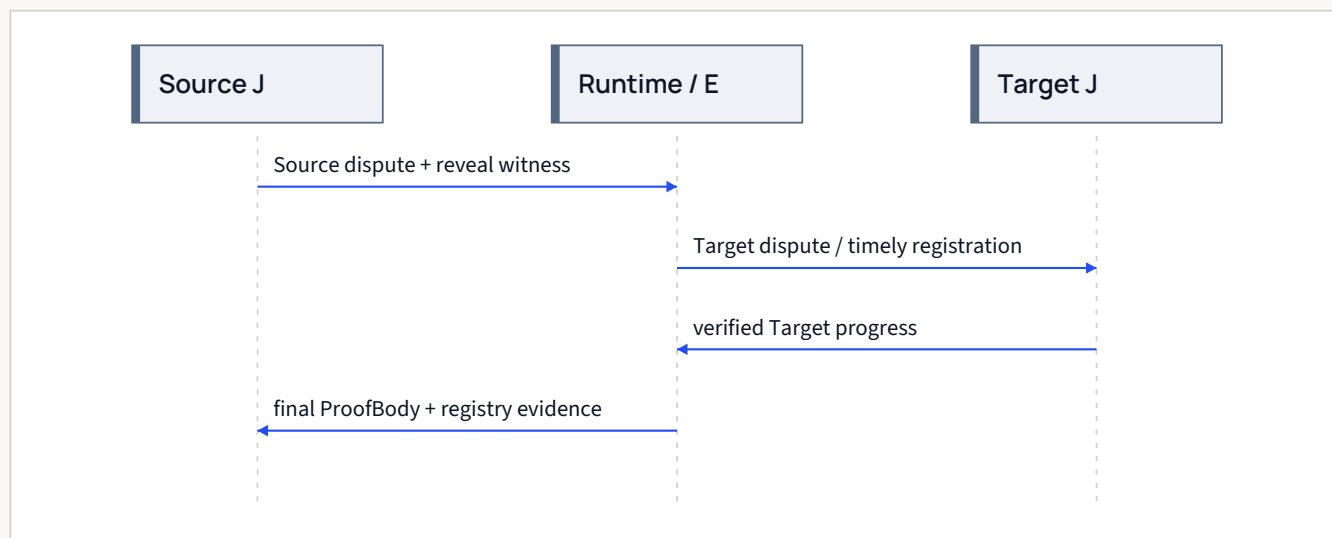


FIG. 24

Логическая связь двух расчётных сред. Публичный reveal появляется при close/dispute, а не для каждого fill.

Почему нужны hash ladders

Частичное исполнение передаёт проверяемую долю через ladder evidence. Signed Pull задаёт exact commitments, role, amount и claimedRatio. J registry хранит evidence в namespace (revealer, counterparty, ladderHash, targetRole). Revealer берётся из outer Hanko; обратная пара Entity - другой slot.

Source и Target имеют разную политику

Source single-shot: exact retry - no-op, другое второе значение запрещено. Target допускает монотонный refresh; меньшая доля запрещена. Settlement читает timestamped registry record, а не произвольный fill argument. Доля quantized: $\text{floor}(\text{amount} \times \text{ratio} / 65535)$. Cooperative matched quote и dispute ladder ratio - разные расчётные условия.

TIMING

Для Account: valid reveal находится в $[S, S + W]$. Если ProofBody содержит Pull, finalization ждёт $T = S + \text{leftResponseSeconds} + \text{rightResponseSeconds}$. У sibling Account собственные S и W; нельзя подставить единые часы для двух J.

CHAPTER 18

Кредит, lending и организация

Новый продукт использует существующие финансовые transitions и полномочия.

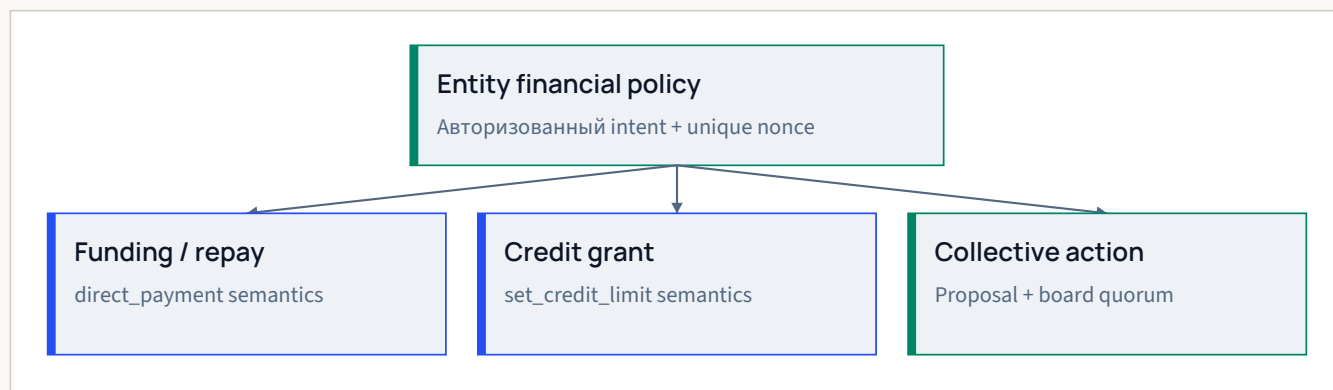


FIG. 25

Lending не вводит вторую формулу баланса. Разные полномочия ведут к каноническим Account handlers.

Кредит выдаёт кредитор

set_credit_limit позволяет proposer предоставить лимит контрагенту: Left меняет rightCreditLimit, Right меняет leftCreditLimit. Лимит задаёт будущий admission; уменьшение не прощает уже подписанную задолженность. Списание долга требует своего явно авторизованного settlement действия.

Funding требует имеющегося актива

lending_fund, repay и payout переиспользуют прямое движение средств. Funding дополнительно исключает возможность представить неиспользованный borrowing limit как внесённый актив: проверка сопоставляет amount + outOwnCredit с outCapacity. Signed lending intent потребляется, чтобы повтор не создал вторую экономическую операцию.

Организация - политика над тем же ядром

Board, collective proposals и command nonce определяют, кто может распорядиться treasury или изменить policy. Это уровень Entity. Размер collateral, credit exposure и денежное изменение остаются уровнем Account. Одна организация может иметь много Accounts без изменения определения финансового инварианта.

```
funding: amount + outOwnCredit <= outCapacity
```

FIG. 26

Неиспользованный кредит исключается из доступного объёма funding.

SOURCE core/account/tx/handlers/balance/lending.ts | core/account/tx/handlers/balance/set-credit-limit.ts | core/entity/consensus/frame/application.ts

CHAPTER 19

J adapters: внешний факт

Watcher превращает подтверждённую историю J в replayable Runtime inputs.

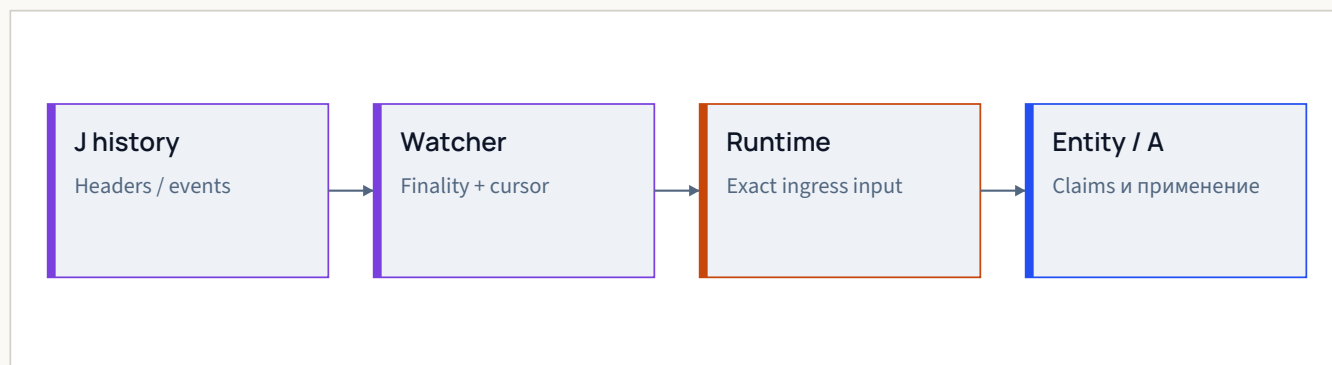


FIG. 27 RPC observation находится за пределами pure transition. Reducer получает аутентифицированную evidence.

От observation к authority

Adapter читает receipts и block history, нормализует J events и выбирает релевантные реплики. Watcher cursor связывается с durable обработкой history range. Entity не должна менять финансовый state по произвольному RPC ответу; её J prefix и Account claims проходят каноническое подтверждение.

Finality определяется конкретной J

Ethereum RPC adapter использует configured confirmation depth; default для chainId 1 - 12. В Tron применяется solidified head SolidityNode, а произвольная ненулевая confirmationDepth запрещена. Это параметры данного adapter, а не универсальная гарантия необратимости для любой сети.

Идентичность стека

Contract addresses берутся из выбранного jurisdiction stack. Asset нельзя объединять только по символу: token id имеет смысл в своём domain. Event duplicate, rewind или range gap требует соответствующего typed path; чтение более свежего head не должно незаметно переписать уже подтверждённую экономическую историю.

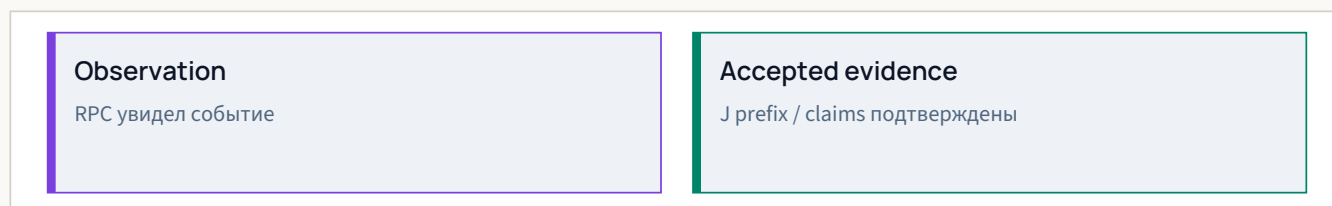


FIG. 28 Увиденный event проходит подтверждение перед финансовым применением.

SOURCE `core/jurisdiction/adapter/watcher/index.ts` | `core/jurisdiction/adapter/rpc/rpc-finality.ts` | `core/jurisdiction/machine/history-consensus/index.ts`

CHAPTER 20

J settlement и sealed batch

Расчётный контракт исполняет ровно те bytes, которые авторизованы Hanko.

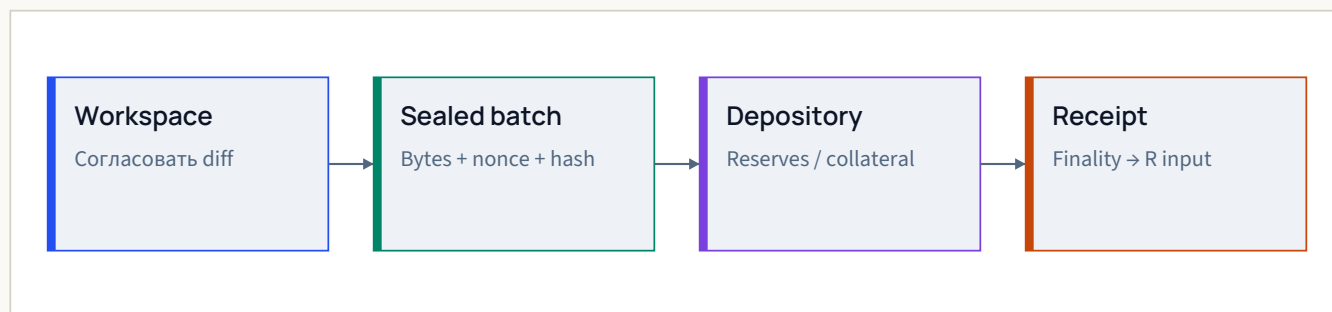


FIG. 29

Submitted transaction ещё не финальный расчёт. Receipt возвращается в машину как проверяемый факт.

Граница external submission

sealed-batch.ts проверяет согласованность batch, encodedBatch, batchHash, batchSize, entityNonce и domain. Adapter не вправе отправить восстановленные bytes, отличающиеся от подписанных. Limits проверяются до submission. J nonce предотвращает повторное исполнение одного авторизованного batch.

Reserves, collateral и debts

Depository хранит расчётные активы Entity и Account collateral. Account.sol вычисляет settlement deltas по signed proof; Depository применяет их к custody records. Если debtor reserves недостаточно, остаток записывается как debt. Этот debt взыскивается из будущих поступлений того же token в этот Depository; внешние активы и юридические требования требуют отдельного основания.

Операция	Смысл
r2c	Reserve proposer → Account collateral
c2r	Account collateral → reserve proposer
r2r	Reserve → reserve контрагента
Dispute finalization	Signed proof → deltas → collateral/reserves/debt

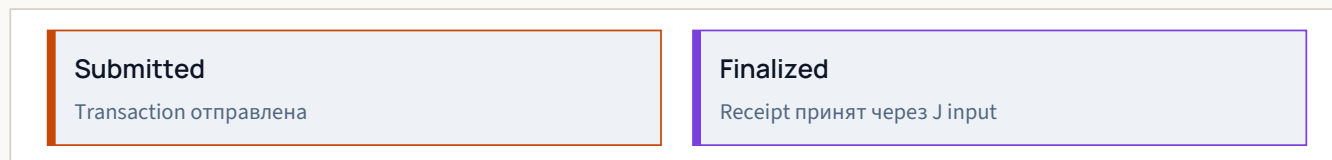


FIG. 30

Submission и финальный расчёт - разные состояния операции.

SOURCE core/jurisdiction/machine/batch/sealed-batch.ts | jurisdictions/contracts/Depository.sol | jurisdictions/contracts/Account.sol

CHAPTER 21

Спор: исполнить доказательство

Неподвижный реер не должен блокировать предусмотренный правилами расчёт.

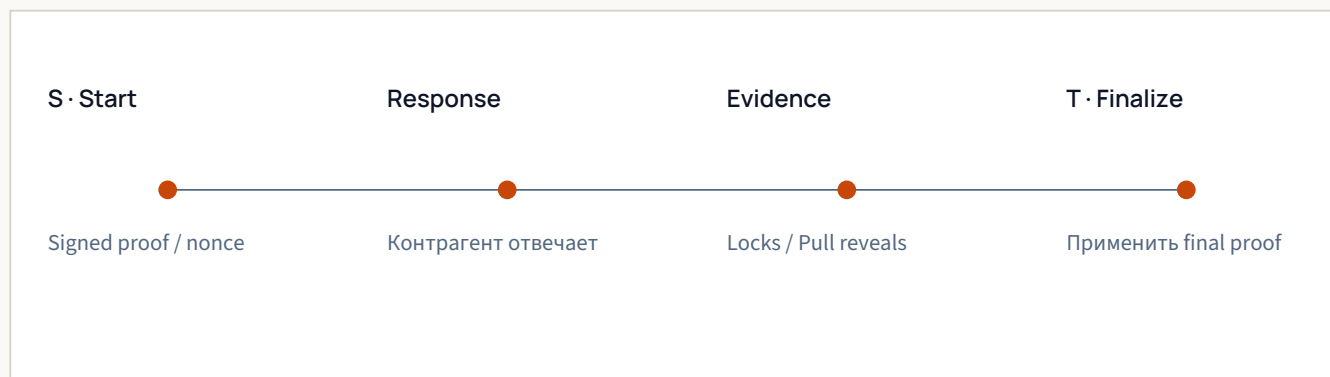


FIG. 31 Точные clocks и допустимые ветви задаются Account dispute config и содержимым ProofBody.

Вход в dispute

Starter подаёт допустимую counterparty evidence, связанную с Account, proof body, proposer side и signed nonce. Preflight заново вычисляет dispute hash и проверяет Hanko в правильном Depository domain. Frame height и dispute nonce - разные величины; их подмена создаёт replay vulnerability.

Ответ и завершение

J учитывает response windows и более свежую допустимую evidence. Finalization строит окончательные deltas, включая условия из proof. Для Pull действует полный reveal barrier. В Pull-free случае немедленная mutual acceptance доступна non-starter с новым outer Hanko; starter без нового согласия контрагента ждёт T.

Какая evidence нужна стороне

Account сохраняет актуальную подписанную proof evidence; history читается отдельно и не сканируется для live settlement. core/watchtower/ содержит backup/recovery и опциональные appointment, dispute sweep и push wake paths. Эти функции требуют настройки и конкретной authority. Online participation и J inclusion до deadline остаются условиями liveness.

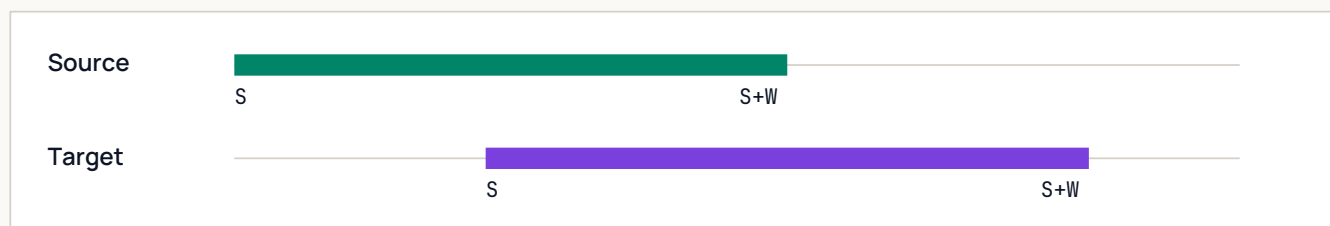


FIG. 32 Source и Target имеют собственные S и W. Окна не обязаны совпадать.

SOURCE core/entity/tx/handlers/dispute/start-hanko.ts | core/protocol/dispute/proof-builder.ts |
core/watchtower/standalone-server.ts | docs/hashladder-registry-spec.md

CHAPTER 22

Delta Transformers

Подписанное условие превращает дополнительные доказательства в ограниченный delta.

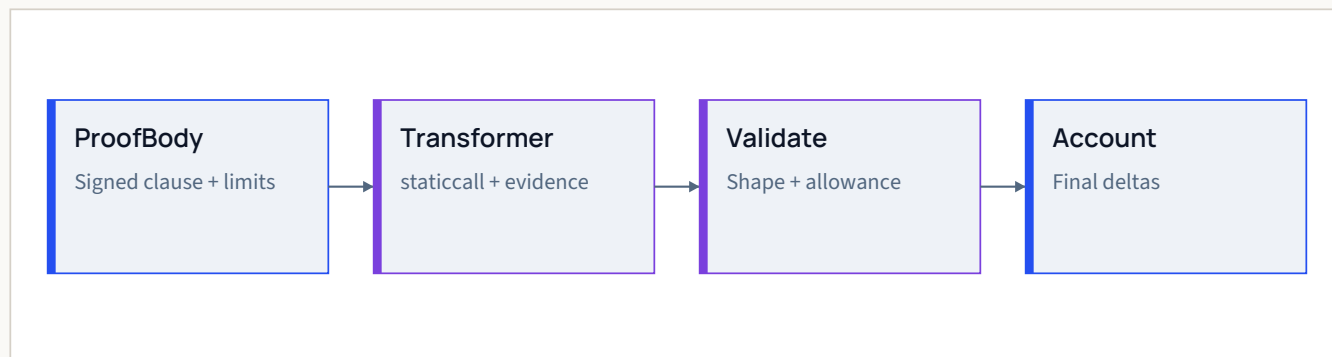


FIG. 33 Transformer вычисляет условный результат. Account владеет окончательной signed delta arithmetic.

Что реализовано в контракте

Account.sol вызывает подписанный transformer через staticcall. Возвращённая структура строго проверяется. Каждый изменённый token требует signed allowance; результат ограничивается её пределами. Reserved post-call gas нужен, чтобы после внешнего вычисления проверить результат и закончить расчёт.

Fail-stop для finalization

Missing code, revert, out-of-gas или malformed output не превращаются в zero delta. Finalization целиком откатывается, а dispute остаётся активным. Это сохраняет смысл подписанного условия: J не вправе подменить неисполнившуюся clause другой финансовой интерпретацией.

Граница текущего продукта

Спецификация counterfactual transformers описывает дальнейший wallet authoring, bilateral clause approval и CREATE2 workflow. Наличие общего контрактного execution path само по себе не доказывает готовность всего такого пользовательского пути. В руководстве эти возможности рассматриваются как направление, пока нет его отдельной end-to-end evidence.

WHY IT MATTERS

Условные обязательства можно исполнять по заранее согласованной программе и evidence. Это расширяет Account, сохраняя authority, token limits и расчётную границу J.

CHAPTER 23

Storage и recovery

После сбоя нужны те же roots и outputs, что и при непрерывной работе.

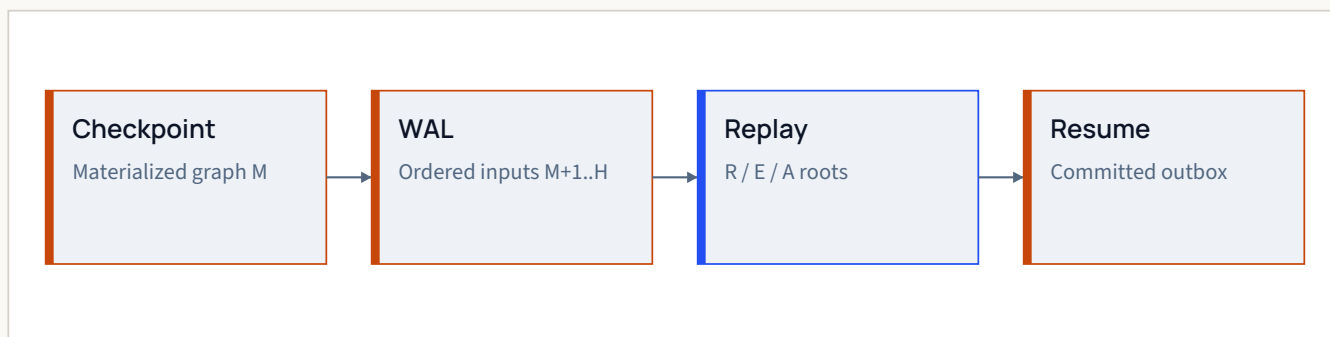


FIG. 34

Восстановление не зависит от утраченного RAM cache или отдельной копии финансового state.

Одна каноническая поверхность

Authoritative LevelDB хранит path-keyed graph records, Runtime WAL, outbox и head. Certified Entity/Account frame history находится в выделенных stores и читается по запросу. Mempool, proposals, votes, retry queues и worker positions не получают самостоятельного durable источника; их необходимая текущая форма выводится replay.

Overlay делает изменения изолированными

Typed overlay читает свои mutations прежде base. Handler работает с отдельными set/delete, не пересоздавая всю Entity или Account. Fold на границе root объединяет dirty paths, сохраняет untouched subtrees и вычисляет missing hashes. Publish и discard принадлежат coordinator; lifecycle token защищает от повторного fold и использования discarded draft.

Как проверять восстановление

Replay берёт checkpoint и упорядоченные accepted Runtime inputs. По каждому frame сравниваются Runtime/Entity/Account roots и ordered event/effect/outbox digests. Подробный Account dump нужен после первого mismatch. Сравнение только итогового balance не заметит промежуточное нарушение authority или порядка effects.

```
recover(graph[M], acceptedInputs[M+1..H]) = live[H]
```

FIG. 35

Сравнение включает roots и ordered outputs, а не только конечный баланс.

CHAPTER 24

Workers и TS / Rust parity

Меняется место исполнения; exact финансовая последовательность остаётся той же.

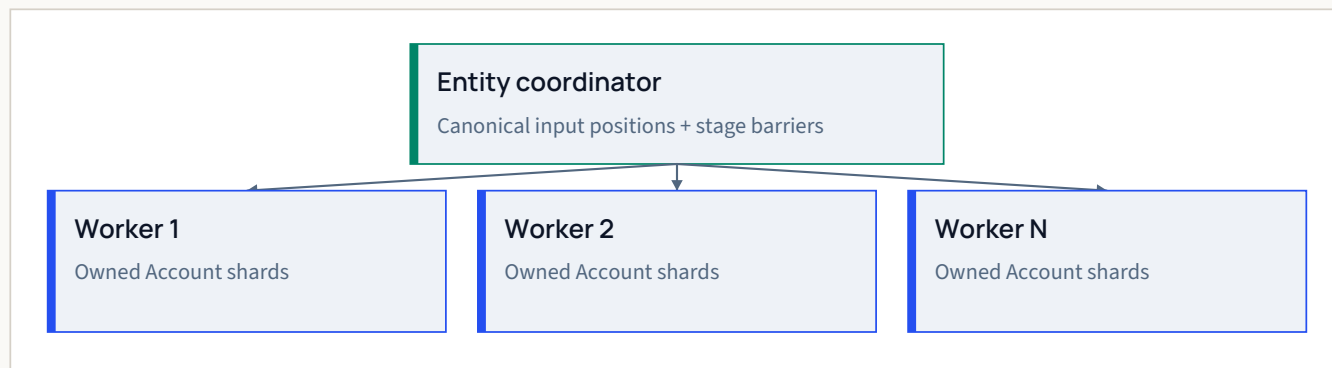


FIG. 36

Join возвращает результаты в исходные slots. Первым закончивший worker не становится первым финансовым output.

Что пересекает boundary

Worker request содержит frame id, controlled timestamp, finalized J height, authority evidence и exact inputs. Result содержит ordered effects, changed subroots и необходимую post-account evidence. Coordinator использует эти результаты в общем Entity candidate. В текущем TS protocol задана полная карта 4096 logical shards на physical workers.

Равенство engines проверяется на одном WAL

Канонический mixed production WAL воспроизводится через TS W1, TS W4, Rust W1 и Rust W4. Проверяется каждый frame и его ordered outputs. Первый divergent frame задаёт единственную ближайшую задачу исправления. Итоговый root без последовательных проверок не доказывает semantic parity.

Replay и live gate отвечают на разные вопросы

Replay доказывает одинаковый переход на уже записанном input. Live J watcher → Entity → batch → receipt проверяет реальную внешнюю границу. Performance claim требует production live path, реального WAL/fsync и machine lock; replay throughput и submitted counts не являются экономическим TPS.



FIG. 37

Join восстанавливает позиции input, прежде чем публиковать effects.

SOURCE [core/rscore/ts-worker/protocol.ts](#) | [core/rscore/ts-worker/coordinator.ts](#) | [AGENTS.md](#)

CHAPTER 25

Проверка и границы гарантий

Обещание системы проверяется на конкретном финансовом и crash boundary.



FIG. 38 Безопасность перехода, восстановление и своевременное исполнение требуют разной evidence.

Малый тест, затем production path

Для новой divergence сначала нужен минимальный vector, затем production-equivalent scenario и повтор исходного артефакта. Проверяются обе стороны Account, exact hash/nonce и порядок outputs. FinTS требует source validation, явных owners и громких invariant failures; TypeScript тип сам по себе не валидирует сетевые bytes.

Что доказательство не исправляет

Подпись делает обязательство проверяемым. Она не создаёт отсутствующий collateral, не устраняет неплатёжеспособность заёмщика и не гарантирует J inclusion до deadline. Credit policy, reserve availability и operational participation остаются частью экономического решения. Прозрачность Account означает проверяемость сторонами; публикация личных финансов всем участникам для этого не требуется.

Evidence этой редакции

Текст сверялся с source tree указанного SHA. В ходе подготовки выполнены 21 focused finance test: 21 pass, 0 fail, 20 653 assertions. Это ограниченная проверка deriveDelta и lending funding, а не release audit. Общий bun run check остановился на отсутствующем cargo в PATH; полный gate не подтверждён.

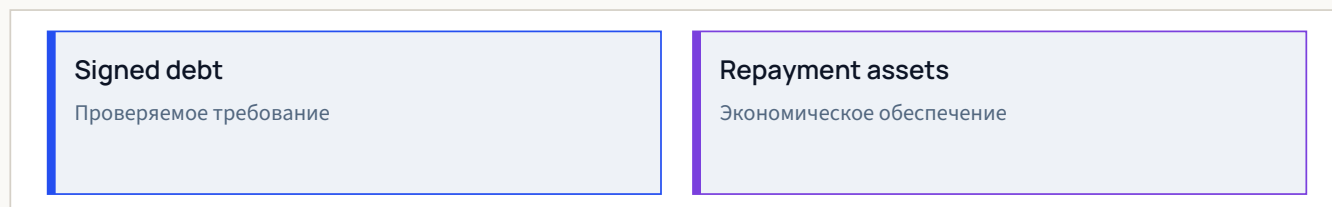


FIG. 39 Проверяемое обязательство не создаёт отсутствующие активы должника.

SOURCE docs/fints.md | core/__tests__/finance/state/derive-delta-property.test.ts |
core/__tests__/finance/state/lending-funding.test.ts

CHAPTER 26

Словарь и маршрут по источникам

Короткие определения и точки входа для следующего чтения кода.

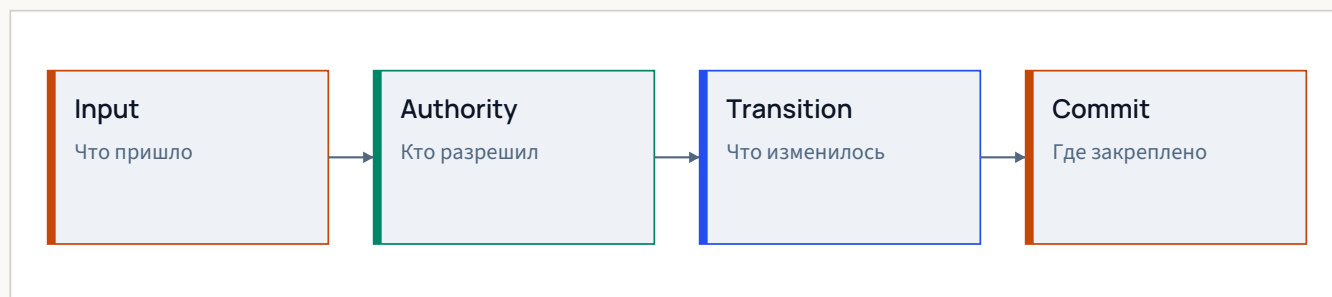


FIG. 40

Четыре вопроса позволяют разобрать любую финансовую операцию xln.

Начните с владельца

Runtime: `core/runtime/frame/process.ts`. Entity: `core/entity/consensus/frame/application.ts` и `input/consensus.ts`.

Account: `core/account/consensus/index.ts`, `tx/apply.ts` и `utils.ts`. J: `core/jurisdiction/adaptor/watcher/index.ts`, `machine/batch/sealed-batch.ts` и `jurisdictions/contracts/Depository.sol`.

Как пользоваться ссылками

SOURCE внизу каждой страницы указывает исходные файлы. PDF outline и оглавление ведут к разделам.

Baseline фиксирует конкретную редакцию; номера строк намеренно не замораживаются. Архитектурный

канон: `docs/core/rjea-architecture.md`. Безопасность TypeScript: `docs/fints.md`. Recovery: `docs/wal.md`. Cross-J clocks: `docs/hashladder-registry-spec.md`.

Термин	Значение
J / E / A	Jurisdiction / Entity / Account
Replica / State	Живая координация / frame-committed данные
Frame / Candidate	Сертифицируемый переход / изолированный результат
Hanko	Проверяемая авторизация Entity для exact digest
RCPAN / Δ / C	Кредитно-обеспеченный Account / позиция / collateral
Hold / allowance	Занятая capacity / signed предел условия
WAL / outbox	Журнал принятых inputs / committed outputs к доставке
Pull / transformer	Условное требование / программа вычисления delta

SOURCE `docs/core/rjea-architecture.md` | `docs/fints.md` | `docs/wal.md`